

Refatoração e Padrão de Projetos

Autor: Ian Dias Duque
Olavo Barros Meira Filho
Davi Goes Bonifacio Berbert Marques
José Vitor Oliveira Patriota Santos

Agenda

O objetivo dessa apresentação é esclarecer a refatoração do projeto

1. Item 1:
Contexto

2. Item 2:
Diagramas

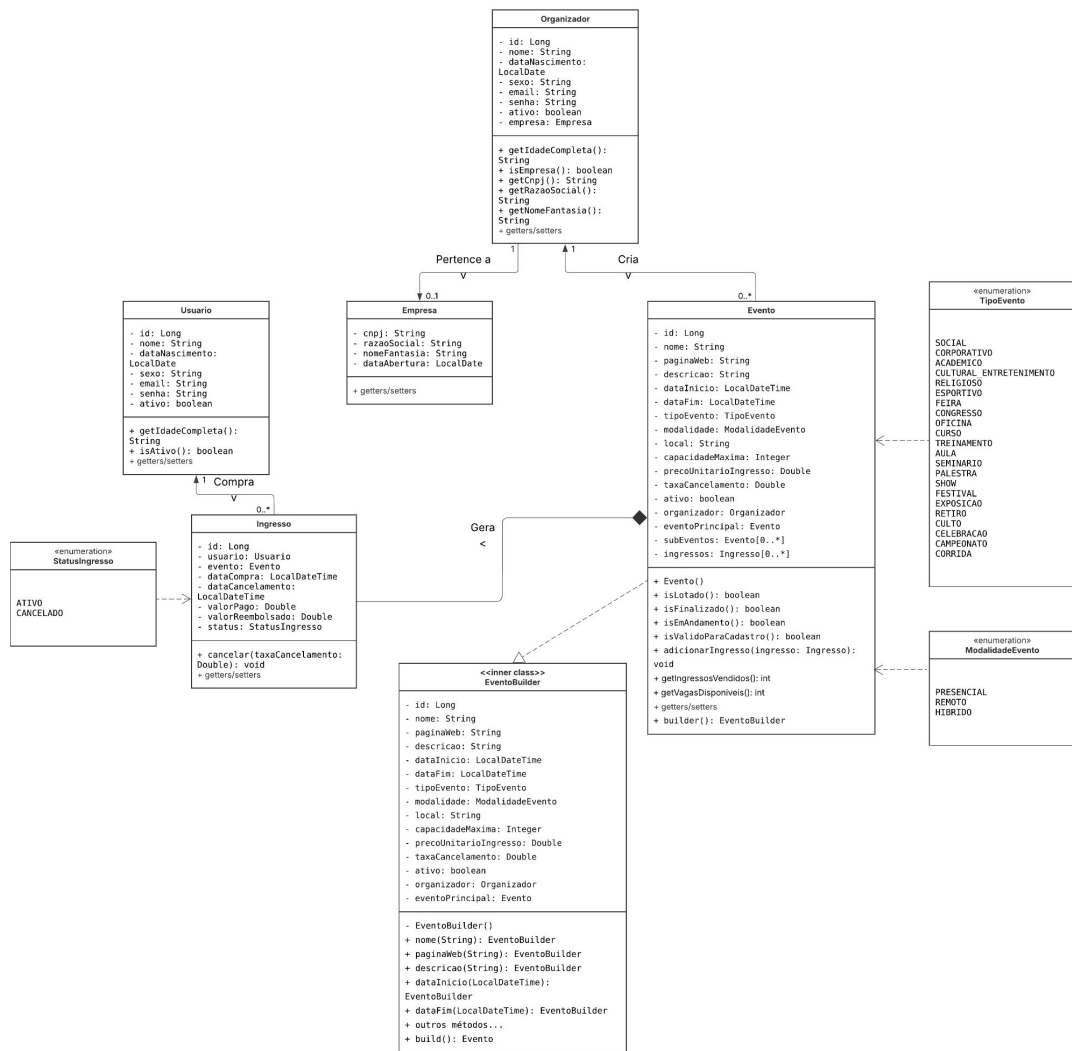
3. Item 3:
Padrões de Projeto

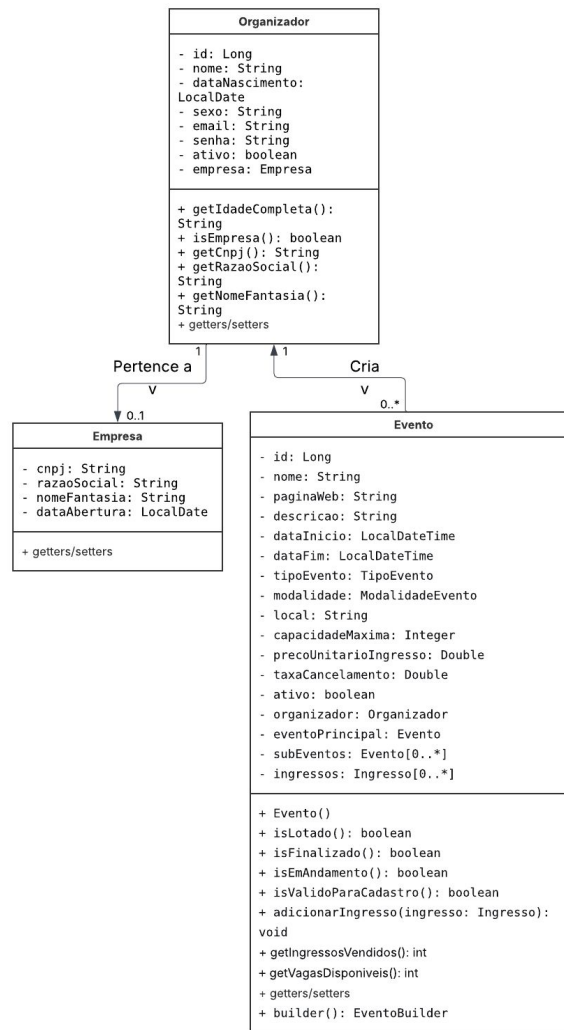
- O objetivo do projeto é o desenvolvimento da nova API do Dendê Eventos, voltado para a organização e gestão de eventos.
- No Desafio 1, foi realizada: Criação das principais classes do sistema; Implementação inicial dos Controllers; Elaboração do diagrama de classes do projeto.
- No Desafio 2, etapa atual do projeto, foram realizadas: Criação dos DTOs; Implementação dos Mappers; Aplicação do padrão Builder na classe Evento; Criação da camada Service; Refatoração dos Controllers; Atualização do diagrama de classes; Criação do diagrama de atividade.

DIAGRAMA DE CLASSES



- Atualização do diagrama de classes, representando as principais entidades do domínio, como Usuário, Organizador, Evento e Ingresso, além de seus atributos, métodos e relacionamentos.





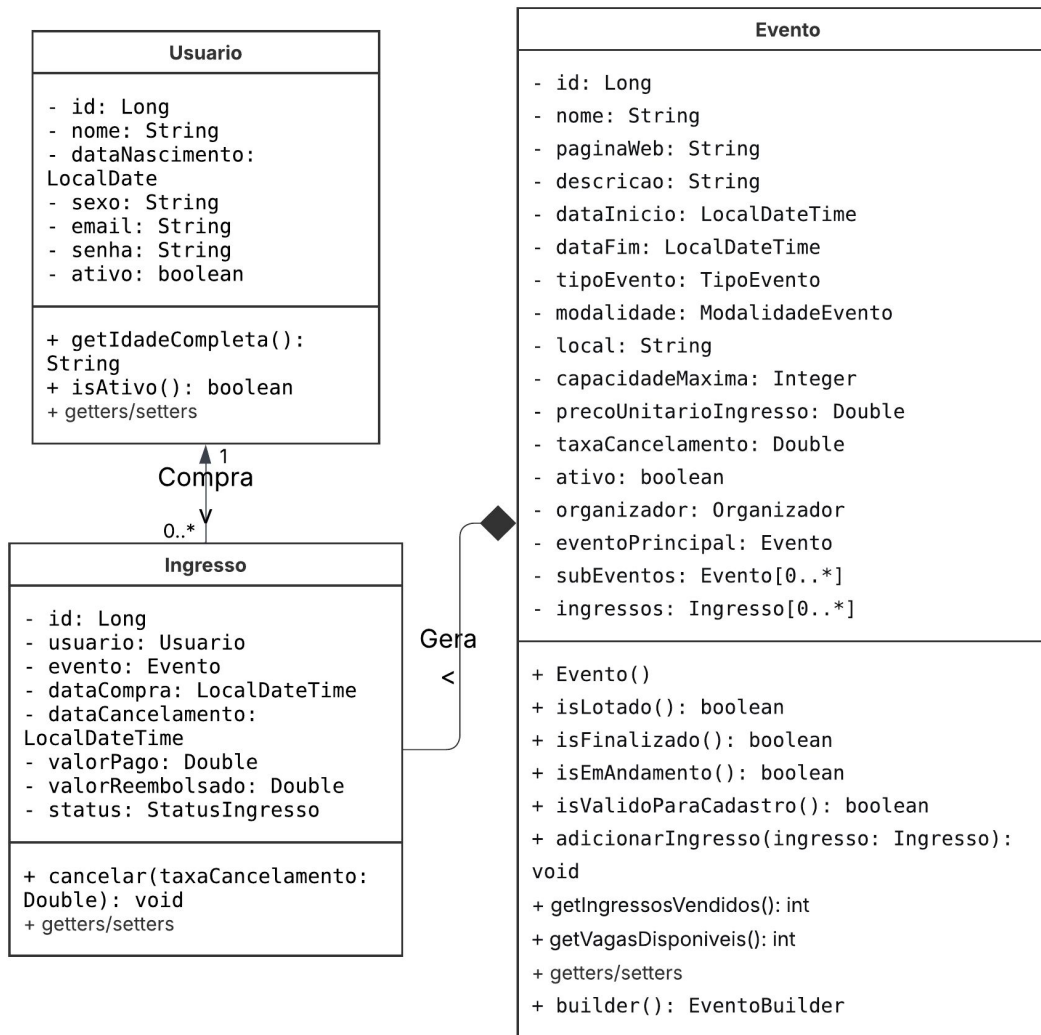
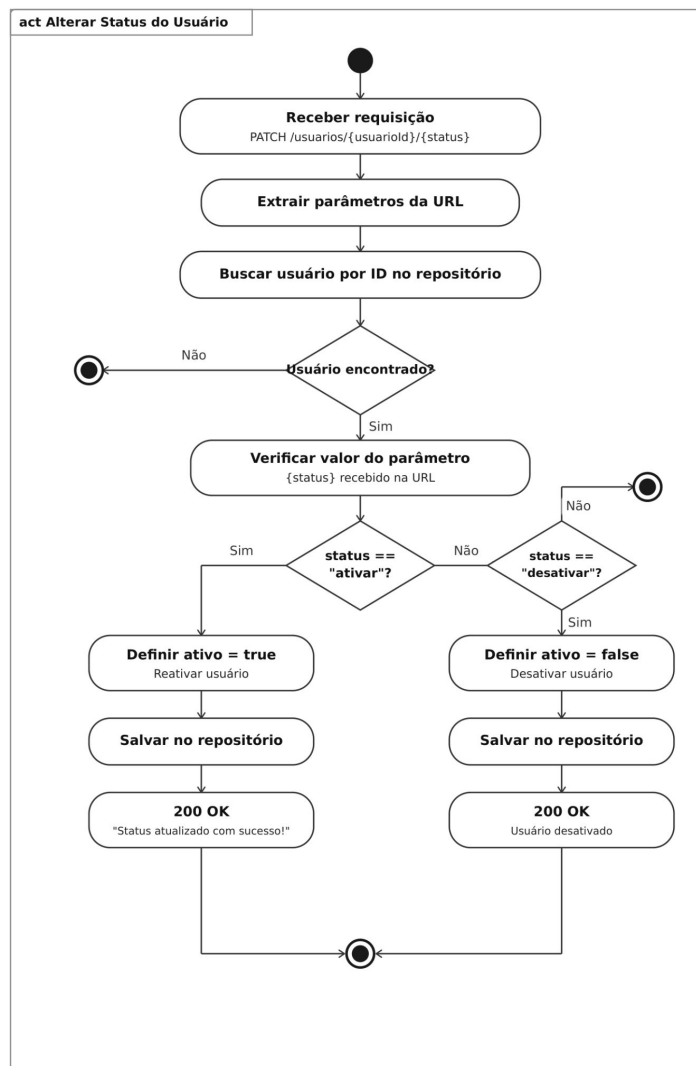


DIAGRAMA DE ATIVIDADE

- Construção do diagrama de atividade referente ao User Story 'REATIVAR USUÁRIO'



PADRÕES DE PROJETO



- Os padrões de projeto são soluções reusáveis para problemas recorrentes.

O objetivo dos padrões de projeto, são tornar componentes reutilizáveis que facilitam a padronização, que permita agilidade para as soluções de problemas recorrentes no desenvolvimento do sistema.

PADRÃO BUILDER

- O Builder é um padrão de projeto criacional, que permite a construção de objetos complexos passo a passo.

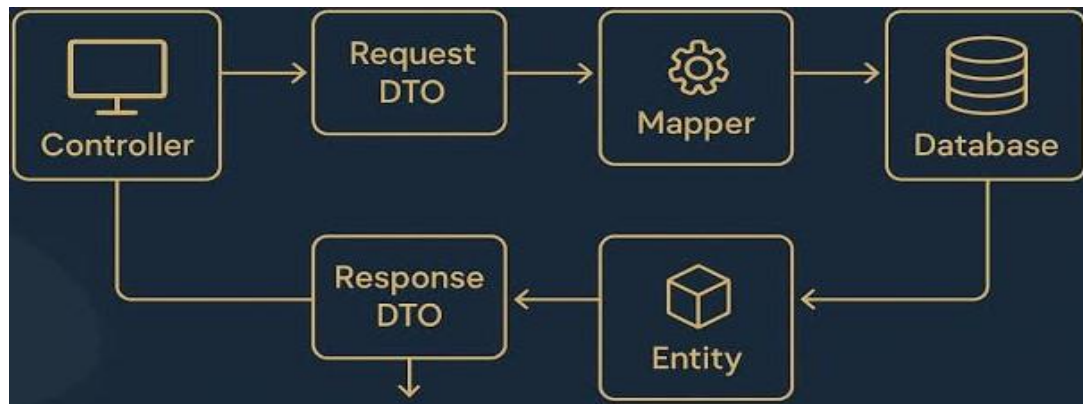
<<inner class>>
EventoBuilder

```
- id: Long  
- nome: String  
- paginaWeb: String  
- descricao: String  
- dataInicio: LocalDateTime  
- dataFim: LocalDateTime  
- tipoEvento: TipoEvento  
- modalidade: ModalidadeEvento  
- local: String  
- capacidadeMaxima: Integer  
- precoUnitarioIngresso: Double  
- taxaCancelamento: Double  
- ativo: boolean  
- organizador: Organizador  
- eventoPrincipal: Evento
```

```
- EventoBuilder()  
+ nome(String): EventoBuilder  
+ paginaWeb(String): EventoBuilder  
+ descricao(String): EventoBuilder  
+ dataInicio(LocalDateTime):  
EventoBuilder  
+ dataFim(LocalDateTime): EventoBuilder  
+ outros métodos...  
+ build(): Evento
```

PADRÃO DTO

- O padrão Data Transfer Object (DTO) é usado para transferir dados entre diferentes camadas de uma aplicação.
- Ele não possui regras de negócio nem métodos que processem dados. Focado em performance e na segurança, protegendo o modelo de domínio. Sua função é apenas transportar dados.
- É aplicado em APIs REST a fim de retornar apenas os campos necessários ao cliente, sem expor a entidade completa do banco de dados.
- É utilizado principalmente quando a entidade de banco tem campos sensíveis, como senhas.



```
public class UsuarioRequestDTO { 9 usages

    private String nome; 2 usages
    private LocalDate dataNascimento; 2 usages
    private String sexo; 2 usages
    private String email; 2 usages
    private String senha; 2 usages

    public UsuarioRequestDTO() {} no usages

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public LocalDate getDataNascimento() { return dataNascimento; } 3 usages
    public void setDataNascimento(LocalDate dataNascimento) { this.dataNascimento = dataNascimento; }

    public String getSexo() { return sexo; } 4 usages
    public void setSexo(String sexo) { this.sexo = sexo; } no usages

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public String getSenha() { return senha; } 6 usages
    public void setSenha(String senha) { this.senha = senha; } no usages
}
```

CAMADAS CONTROLLER, SERVICE, **unex** REPOSITORY

- O Controller é a "porta de entrada" da aplicação. Sua função é lidar com o protocolo de comunicação.
- O Service é o "cérebro" da aplicação. Ele visa abstrair regras de negócio das rotas, além de tornar o código mais reutilizável.
- O Repository é a ponte entre a aplicação e a fonte de dados, seja ela um banco de dados, um arquivo físico ou qualquer outro meio de persistência de dados da aplicação.

CONTROLLER



- O Controller é responsável por receber as requisições HTTP da API e retornar a resposta ao cliente. Ele identifica o método HTTP utilizado (GET, POST, PUT, PATCH) e extrai os dados da requisição.
- Responsável por receber requisições HTTP, delegar a lógica de negócios para a camada Service e retornar respostas usando um DTO. Define ou mapeia as rotas da API (endpoints), associando cada URL a um método específico.
- Aplica o controle de acesso, garantindo que apenas usuários com as permissões corretas possam acessar determinados endpoints.

CONTROLLER



```
public class UsuarioController {

    @GetMapping(path =("/{usuarioId}") no usages
    public ResponseEntity<Object> visualizarPerfil(
        @PathVariable(parameter = "usuarioId") long usuarioId) {
        try {
            UsuarioResponseDTO response = usuarioService.buscarPorId(usuarioId);
            return ResponseEntity.ok(response);
        } catch (IllegalArgumentException e) {
            return ResponseEntity.ok( body: "ERRO: " + e.getMessage());
        }
    }

    @PostMapping(path =("/{usuarioId}/{status}") no usages
    public ResponseEntity<Object> alterarStatus(
        @PathVariable(parameter = "usuarioId") long usuarioId,
        @PathVariable(parameter = "status") String status) {
        try {
            usuarioService.alterarStatus(usuarioId, status);
            return ResponseEntity.ok( body: "Status do usuário atualizado com sucesso!");
        } catch (IllegalArgumentException e) {
            return ResponseEntity.ok( body: "ERRO: " + e.getMessage());
        }
    }
}
```

MAPPER



- O Mapper é utilizado para converter objetos DTO em entidades e vice-versa.
- **Por que usar?** O uso do Mapper promove a separação de responsabilidades, automatiza a conversão de dados entre objetos (por exemplo, de Entidade para DTO), reduz código repetitivo e evita que a API exponha diretamente as entidades do banco de dados.
- **Quando usar?** Sempre que houver necessidade de trafegar dados entre camadas distintas da aplicação, especialmente entre Controller, Service e Repository, utilizando DTOs para entrada e saída de dados.

MAPPER



```
public class UsuarioMapper { 6 usages

    private UsuarioMapper() { no usages

    }

    public static Usuario toModel(UsuarioRequestDTO dto) { 1 usage
        if (dto == null) return null;

        Usuario usuario = new Usuario();
        usuario.setNome(dto.getNome());
        usuario.setDataNascimento(dto.getDataNascimento());
        usuario.setSexo(dto.getSexo());
        usuario.setEmail(dto.getEmail());
        usuario.setSenha(dto.getSenha());
        return usuario;
    }
```

CONSIDERAÇÕES FINAIS



- O desenvolvimento desta etapa do projeto permitiu aprimorar a organização e a estrutura da aplicação, aplicando padrões de projeto e separação de responsabilidades entre as camadas do sistema.
- O projeto foi estruturado de forma que possa evoluir ao longo dos próximos desafios propostos na disciplina, permitindo a implementação de novas funcionalidades e melhorias conforme as próximas demandas do sistema Dendê Eventos.

TRASEL, Robson. *Usando padrão de projeto Builder com Java*. Disponível em:
<https://medium.com/@robson.trasel/usando-padr%C3%A3o-de-projeto-builder-com-java-6b7001310e6d>

DIO. *O que é o padrão Builder (Java/Spring Boot)*. Disponível em:
<https://www.dio.me/articles/o-que-e-o-padrao-builder-javaspring-boot>

DIO. *DTO (Data Transfer Object)*. Disponível em:
<https://www.dio.me/articles/dto-data-transfer-object>

TREINAWEB. *Spring Boot: trabalhando com controllers*. Disponível em:
<https://www.treinaweb.com.br/blog/spring-boot-trabalhando-com-controllers>

DIO. *O que é o Repository*. Disponível em:
<https://www.dio.me/articles/o-que-e-o-repository-866d13a8e1e5>